

AI ASSISTED CODING

LAB-16: Database Design and Queries: Schema Design and SQL Generation

Roll no: 2503A5L11

Name: P. Susmija

Batch: 25BTCAICSB19

Task-1 Description: Schema Generation

Task: Ask AI to design a schema for a Library Management System (Tables: Books, Members, Loans).

Prompt: Design a schema for a Library Management System (Tables: Books, Members, Loans).

Code Generated:

```
task1-16.sql
> Run on active connection | Select block
1 CREATE TABLE Members (
2     member_id INT PRIMARY KEY,
3     name VARCHAR(100),
4     email VARCHAR(100) UNIQUE,
5     join_date DATE
6 );
7
8 CREATE TABLE Books (
9     book_id INT PRIMARY KEY,
10    title VARCHAR(200),
11    author VARCHAR(100),
12    available BOOLEAN
13 );
14
15 CREATE TABLE Loans (
16     loan_id INT PRIMARY KEY,
17     member_id INT,
18     book_id INT,
19     loan_date DATE,
20     return_date DATE,
21     FOREIGN KEY (member_id) REFERENCES Members(member_id),
22     FOREIGN KEY (book_id) REFERENCES Books(book_id)
23 );
24
```

Observation: The AI successfully designed the Library Management System schema with the required tables and appropriate attributes to manage books, members, and loan details. It ensured proper structure that supports future query execution and relationships between tables. This task shows that AI can quickly and accurately create a complete database schema.

Task-2 Description: Error Insert Data

Task: Ask AI to generate INSERT INTO queries for the schema above (3 sample records per table).

Prompt: Generate INSERT INTO queries for the schema above (3 sample records per table).

Code Generated:

```
task2-16.sql
-- Run on active connection | Select block
-- Sample data for Members (3 records)
INSERT INTO Members (member_id, name, email, join_date) VALUES
(1, 'Alice Johnson', 'alice.j@example.com', '2023-01-15'),
(2, 'Bob Smith', 'bob.smith@example.com', '2022-06-10'),
(3, 'Carol Nguyen', 'carol.ng@example.com', '2021-11-05');

-- Sample data for Books (3 records)
INSERT INTO Books (book_id, title, author, available) VALUES
(101, 'The Pragmatic Programmer', 'Andrew Hunt', TRUE),
(102, 'Clean Code', 'Robert C. Martin', TRUE),
(103, 'Introduction to Algorithms', 'Cormen et al.', FALSE);

-- Sample data for Loans (3 records)
INSERT INTO Loans (loan_id, member_id, book_id, loan_date, return_date) VALUES
(1001, 1, 101, '2024-09-01', '2024-09-15'),
(1002, 2, 103, '2024-09-10', NULL),
(1003, 3, 102, '2024-08-20', '2024-09-02');

SELECT * FROM Members;
SELECT * FROM Books;
SELECT * FROM Loans;
```

Output:

Local Postgres: multiple query results

< (3 records) INSE... -- Sample data for Loans (3 records) INSE...

SELECT * FROM Members;

SELECT * FROM Books;

member_id	name	email	join_date	status
1	Alice Johnson	alice.j@example.com	2023-01-15	active
2	Bob Smith	bob.smith@example.co...	2022-06-10	active
3	Carol Nguyen	carol.ng@example.com	2021-11-05	active

Local Postgres: multiple query results

< (3 records) INSE... -- Sample data for Loans (3 records) INSE...

SELECT * FROM Members;

SELECT * FROM Books;

book_id	title	author	available
101	The Pragmatic Programmer	Andrew Hunt	TRUE
102	Clean Code	Robert C. Martin	TRUE
103	Introduction to Algorithms	Cormen et al.	FALSE

Local Postgres: multiple query results

< (3 records) INSE... -- Sample data for Loans (3 records) INSE...

SELECT * FROM Members;

SELECT * FROM Books;

SELECT * FROM Loans;

loan_id	member_id	book_id	loan_date	return_date
1001	1	101	2024-09-01	2024-09-15
1002	2	103	2024-09-10	NULL
1003	3	102	2024-08-20	2024-09-02

Observation: AI generated correct INSERT INTO statements with sample data for each of the tables created in Task-1. The execution of queries indicates proper syntax and ensures that the database has relevant test records. This task highlights AI’s usefulness in saving time while populating databases with meaningful sample data.

Task-3 Description: Basic Queries

Task: Use AI to generate a query to list all books borrowed by a specific member.

Prompt: Generate a query to list all books borrowed by a specific member.

Code Generated:

```
task3-16.sql
D>Run on active connection |≡Select block
1  -- Query: List all books borrowed by a specific member (use member_id)
2  -- Replace :member_id with the actual id or use your DB's parameter placeholder
3  SELECT
4      l.loan_id,
5      l.book_id,
6      b.title,
7      b.author,
8      l.loan_date,
9      l.return_date,
10     CASE WHEN l.return_date IS NULL THEN 'out' ELSE 'returned' END AS status
11 FROM Loans l
12 JOIN Books b ON l.book_id = b.book_id
13 WHERE l.member_id = :member_id;
14
15 -- Query: List all books borrowed by a member by name (if you only have the name)
16 -- Replace :member_name with the member's full name (or parameter placeholder)
17 SELECT
18     l.loan_id,
19     m.member_id,
20     m.name,
21     b.book_id,
22     b.title,
23     b.author,
24     l.loan_date,
25     l.return_date,
26     CASE WHEN l.return_date IS NULL THEN 'out' ELSE 'returned' END AS status
27 FROM Loans l
28 JOIN Books b ON l.book_id = b.book_id
29 JOIN Members m ON l.member_id = m.member_id
30 WHERE m.name = :member_name;
31
32 -- Query: Currently borrowed books for a given member (not yet returned)
33 SELECT
34     l.loan_id,
35     b.book_id,
36     b.title,
37     b.author,
38     l.loan_date
39 FROM Loans l
40 JOIN Books b ON l.book_id = b.book_id
41 WHERE l.member_id = 2
42     AND l.return_date IS NULL
43 ORDER BY l.loan_date;
44
```

Output:

Local Postgres: multiple query results X						
-- Query: List all books borrowed by a spe...			-- Query: List all books borrowed by a me...		-- Query: Currently borrowed books for a ...	
loan_id	book_id	title	author	loan_date	return_date	status
 Filter...	 Filter...	 Filter...	 Filter...	 Filter...	 Filter...	 Filter...
1001	101	The Pragmatic Programmer	Andrew Hunt	2024-09-01	2024-09-15	returned

Local Postgres: multiple query results X								
-- Query: List all books borrowed by a spe...			-- Query: List all books borrowed by a me...			-- Query: Currently borrowed books for a ...		
loan_id	member_id	name	book_id	title	author	loan_date	return_date	status
Filter...	Filter...	Filter...	Filter...	Filter...	Filter...	Filter...	Filter...	Filter...
1001	1	Alice Johnson	101	The Pragmatic Programmer	Andrew Hunt	2024-09-01	2024-09-15	returned

Local Postgres: multiple query results X					
-- Query: List all books borrowed by a spe...		-- Query: List all books borrowed by a me...		-- Query: Currently borrowed books for a ...	
loan_id	book_id	title	author	loan_date	
Filter...	Filter...	Filter...	Filter...	Filter...	
1002	103	Introduction to Algorithms	Cormen et al.	2024-09-10	

Observation: The AI provided an accurate SQL SELECT query to list books borrowed by a specific member, using table relationships effectively. It ensures correct retrieval of data from multiple tables, showing the practical application of the designed schema. This demonstrates how AI helps in generating precise queries for real-time information access.

Task-4 Description: Update and Delete Queries

Task: Generate queries with AI for:

- Updating a book's availability to FALSE when borrowed.
- Deleting a member record safely.

Prompt: Generate queries for:

- Updating a book's availability to FALSE when borrowed.
- Deleting a member record safely.

Code Generated:

```
task4-16.sql
-- Run on active connection | =Select block
1  -- === Only two query groups below ===
2  -- 1) Update a book's availability to FALSE when borrowed (transactional, safe)
3  -- Literal runnable example (change ids as needed):
4  BEGIN;
5      SELECT available FROM Books WHERE book_id = 101 FOR UPDATE;
6      UPDATE Books SET available = FALSE WHERE book_id = 101 AND available = TRUE;
7      INSERT INTO Loans (loan_id, member_id, book_id, loan_date, return_date)
8      VALUES (3001, 2, 101, CURRENT_DATE, NULL);
9  COMMIT;
10
11 -- Single-statement guarded CTE (Postgres): atomic update+insert if book available
12 WITH locked AS (
13     SELECT book_id FROM Books WHERE book_id = 102 AND available = TRUE FOR UPDATE
14 ), upd AS (
15     UPDATE Books SET available = FALSE
16     WHERE book_id IN (SELECT book_id FROM locked)
17     RETURNING book_id
18 )
19 INSERT INTO Loans (loan_id, member_id, book_id, loan_date, return_date)
20 SELECT 3002, 3, book_id, CURRENT_DATE, NULL FROM upd;
21
22 -- 2) Safely delete a member record
23 -- Literal runnable example (change member_id as needed):
24 DELETE FROM Members m
25 WHERE m.member_id = 5
26    AND NOT EXISTS (
27     SELECT 1 FROM Loans l WHERE l.member_id = m.member_id AND l.return_date IS NULL
28 );
29
30 -- Option B: Soft-delete (preserve history):
31 ALTER TABLE IF EXISTS Members ADD COLUMN IF NOT EXISTS status VARCHAR(20) DEFAULT 'active';
32 UPDATE Members SET status = 'deleted' WHERE member_id = 6;
33
```

```
16retrieve.sql
  ▶ Run on active connection | ≡ Select block
  -- Retrieval queries to inspect the current state after updates
2
3
4 -- 1) All books with availability
5 SELECT book_id, title, author, available FROM Books ORDER BY book_id;
6
7 -- 2) All loans (history)
8 SELECT loan_id, member_id, book_id, loan_date, return_date FROM Loans ORDER BY loan_date DESC;
9
10 -- 3) Active (currently out) loans
11 SELECT l.loan_id, l.member_id, m.name, l.book_id, b.title, l.loan_date
12 FROM Loans l
13 JOIN Members m ON l.member_id = m.member_id
14 JOIN Books b ON l.book_id = b.book_id
15 WHERE l.return_date IS NULL
16 ORDER BY l.loan_date;
17
18 -- 4) Active loans for a specific member (literal example member_id = 2)
19 SELECT l.loan_id, b.book_id, b.title, l.loan_date
20 FROM Loans l JOIN Books b ON l.book_id = b.book_id
21 WHERE l.member_id = 2 AND l.return_date IS NULL;
22
23 -- 5) Members marked as 'deleted' (soft-deleted)
24 SELECT member_id, name, email, status FROM Members WHERE status = 'deleted';
25
26 -- 6) Books that are currently unavailable
27 SELECT book_id, title, author FROM Books WHERE available = FALSE;
28
29 -- 7) Check specific loan ids inserted by example operations
30 SELECT loan_id, member_id, book_id, loan_date, return_date FROM Loans WHERE loan_id IN (1002, 3001, 3002, 2002, 1004);
```

Output:

Local Postgres: multiple query results			
-- Retrieval queries to inspect the current ...			
book_id	title	author	available
101	The Pragmatic Programmer	Andrew Hunt	FALSE
102	Clean Code	Robert C. Martin	FALSE
103	Introduction to Algorithms	Cormen et al.	FALSE

Local Postgres: multiple query results				
-- 2) All loans (history) SELECT loan_id, m...				
loan_id	member_id	book_id	loan_date	return_date
3001	2	101	2025-10-29	NULL
3002	3	102	2025-10-29	NULL
1002	2	103	2024-09-10	NULL
1001	1	101	2024-09-01	2024-09-15
1003	3	102	2024-08-20	2024-09-02

Local Postgres: multiple query results					
-- 3) Active (currently out) loans SELECT l...					
loan_id	member_id	name	book_id	title	loan_date
1002	2	Bob Smith	103	Introduction to Algorithms	2024-09-10
3001	2	Bob Smith	101	The Pragmatic Programmer	2025-10-29
3002	3	Carol Nguyen	102	Clean Code	2025-10-29

Local Postgres: multiple query results			
-- 3) Active (cur			
loan_id	book_id	title	loan_date
1002	103	Introduction to Algorithms	2024-09-10
3001	101	The Pragmatic Programmer	2025-10-29

Local Postgres: multiple query results X

< loans SELECT I... -- 4) Active loans for a specific member (li... -- 5) Members marked as 'deleted' (soft-d... -- 6) Books that are cur

member_id	name	email	status
Filter...	Filter...	Filter...	Filter...
No data			

Local Postgres: multiple query results X

< loans SELECT I... -- 4) Active loans for a specific member (li... -- 5) Members marked as 'deleted' (soft-d...

book_id	title	author
Filter...	Filter...	Filter...
103	Introduction to Algorithms	Cormen et al.
101	The Pragmatic Programmer	Andrew Hunt
102	Clean Code	Robert C. Martin

Local Postgres: multiple query results X

< loans SELECT I... -- 4) Active loans for a specific member (li... -- 5) Members marked as 'deleted' (soft-d... -- 6) Books that are current

loan_id	member_id	book_id	loan_date	return_date
Filter...	Filter...	Filter...	Filter...	Filter...
1002	2	103	2024-09-10	NULL
3001	2	101	2025-10-29	NULL
3002	3	102	2025-10-29	NULL

Observation: AI generated appropriate UPDATE and DELETE queries to manage changes in the database, including updating book availability and removing member details safely. The commands reflect secure handling of data modifications without affecting integrity. This task proves that AI can assist in maintaining and controlling database operations efficiently.